Digital Information Processing Lab 2025

Class 2: Spectral representation and signal windowing

Group 14 : Snoopy

06.06.2025

Name	Mat. Nr.	Email
Yash Khiste	254751	yash.khiste@st.ovgu.de
Saiful Islam	253574	saiful1.islam@st.ovgu.de
Supritha Rajashekaraiah	244941	supritha.rajashekaraiah@ovgu.de

Class 2: Spectral Representation and Signal Windowing

Preparatory Tasks

1. How is the sampling frequency defined?

The number of samples taken per second from a continuous signal to make it discrete is called the sampling frequency (f_s), and it is also referred to as the sampling rate.This is guided by the Nyquist-Shannon Sampling Theorem, which states: "A signal can be exactly reconstructed from its samples if it is band-limited and the sampling frequency is at least twice the maximum frequency present in the signal." T is the time interval between two sample points. [1]

$$f_s = 1/T$$

2. What happens to the output signal when under/oversampling?

According to Nyquist-Shannon sampling theorem: $\Omega_s \geq 2\Omega_{\mathrm{N}}$

Ω_s : Sampling frequency

Ω_N : Orginal singal frequecny.

Undersampling : $\Omega_s < 2\Omega_N$, If sampling frequency is less than the signal frequency. As depicted in Fig_1(d), it can be seen that the spectrum of the sampled signal is overlapping,

which is also called aliasing. That is, the information of the signal is misinterpreted. When this signal is passed through a low-pass filter for reconstruction, it cannot be completely recovered as the original signal. Aliasing may also occur when the signal is not band-limited.

Oversampling: $\Omega_s >> 2\Omega_N$,If sampling frequency is greater than the signal frequency. As seen in the previous exercise, this increases the precision of the reconstructed signal. As shown in Fig_1(c) the width of ($\Omega_s - 2\Omega_N$) increases significantly, which prevents the signal from overlapping. This eliminates the aliasing effect. It also helps in designing an analog anti-aliasing filter before the ADC and reduces the ADC noise floor through possible noise shaping, allowing a low-resolution ADC to be used. [2]

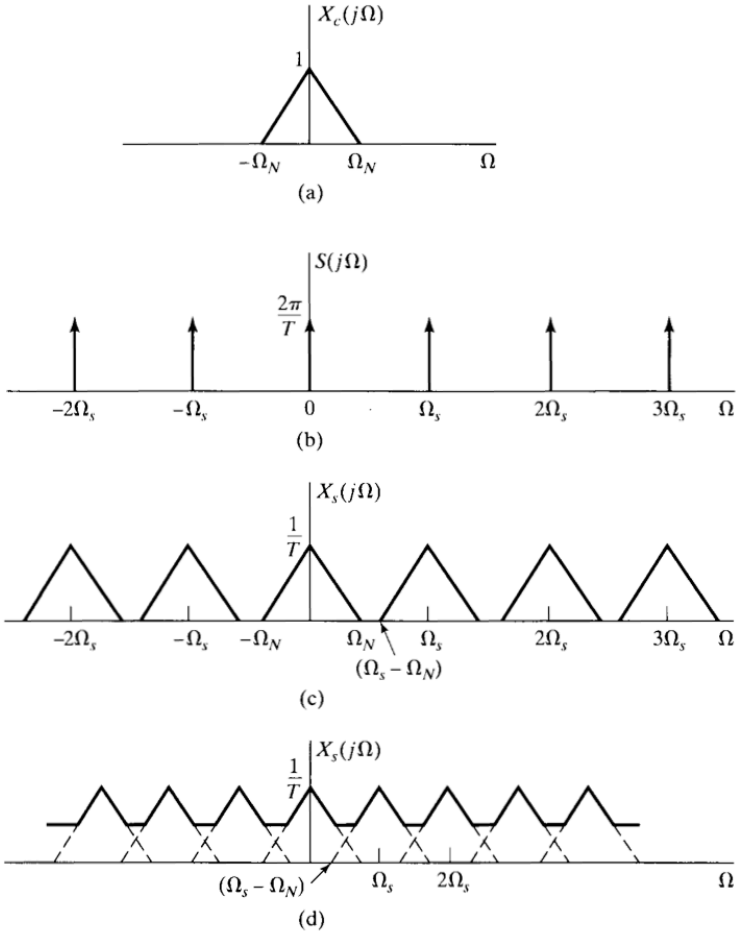


Fig_1: Sampling Cases in frequency domain. (a) Spectrum of a original signal. (b) Spectrum of a sampling function. (c) Spectrum of sampled signal with (Ω_s > 2Ω_N). (d) Spectrum of sampled signal with (Ω_s < 2Ω_N). [1]

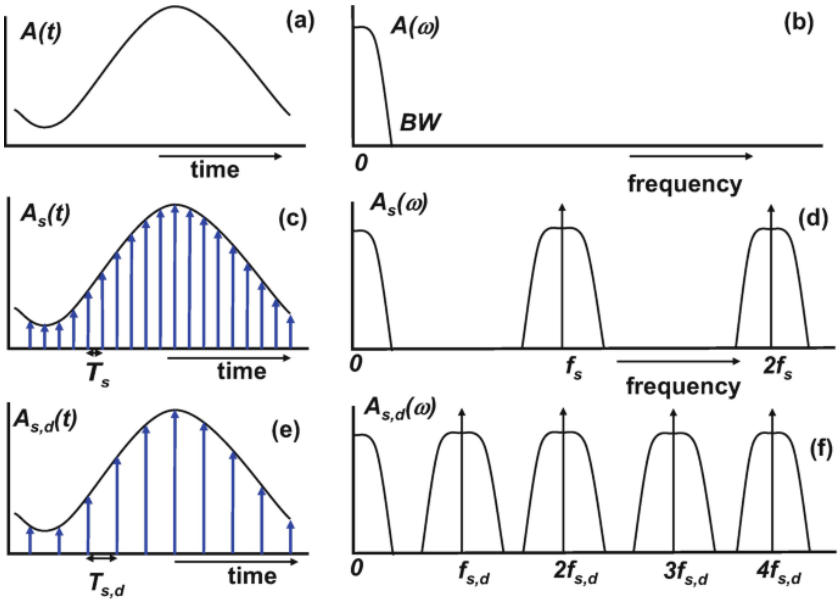


Fig_2: Sampling Cases in time and frequency domain. [3]

3. How do you determine the sampling frequency for a given signal? What happens during superposition of multiple signals?

a) The sampling frequency for any given signal should be determined in such a way that the reconstructed signal captures all the essential information without distortion or loss of data. [1]

Mathematically: $\Omega_s \geq 2\Omega_{\max}$

- Ω_s is the sampling frequency,
- $\Omega_{\max}$ is the highest frequency component in the signal.

b) When multiple signals are combined, the Superposition Principle in signal processing is demonstrated—meaning that the response caused by multiple inputs is the sum of the responses that would have been caused by each input individually. When multiple signals are combined:

- The resulting signal contains all frequency components from the individual signals.
- The highest frequency component among all signals dictates the necessary sampling rate. [1]

4. List a few different windowing functions and explain which function is useful for signal analysis in the frequency domain. What happens to the amplitudes in the time domain?

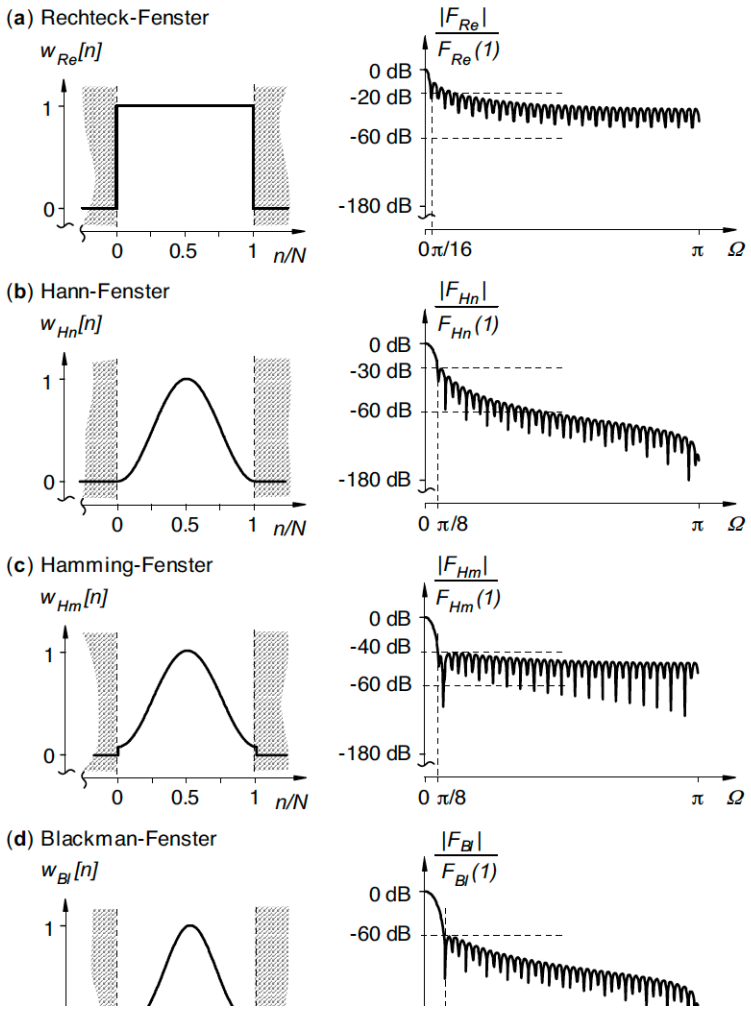
The procedure of taking a small portion of a larger data, for processing and analysis is called windowing. It is done by using a window function or a tapering function. Here are some window functions are listed below:

a) Rectangular window.

b) Hann window.

c) Hamming window.

d) Blackman window.



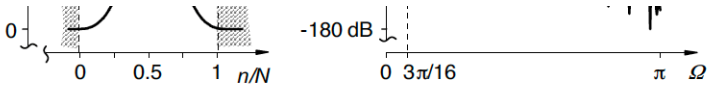


Fig_3: Different window sequences in the time and frequency domain. [4]

To analyze signal in the frequency domain, the Hann window function could be the wise

choice. Here are some reason:

- Reduces spectral leakage(side lobes ≈ -30 dB).
- Better attenuation at stop-band.
- Maintain decent frequency resolution.
- Stops discontinuities at signal edges by smooth tapering.

In the time domain, windowing attenuates the signal at the edges. Tapering modifies the signal amplitude at the beginning and end by gradually reducing the values, which makes the signal go to the zero smoothly.

A windowed signal is resulted by applying a window $w[n]$ to a time-domain signal $x[n]$:

$$x_w\,[n] = x[n] \cdot w[n]$$

This multiplication decreases the amplitude of the signal, explicitly at the edges, as most window functions w[n] have values less than 1 (except at or near the center). So, the overall amplitude of the windowed signal stays lower than the original signal. If it is not properly compensated, this reduction can lead to attenuated spectral amplitudes in the frequency domain

References

1. ^Oppenheim, A. & Schafer, R. (2013). Discrete-Time Signal Processing. (3rd ed.). Pearson International. https://elibrary.pearson.de/book/99.150005/9781292038155
2. ^ Tan, Li; Digital Signal Processing Fundamentals and Application. San Diego : Elsevier Science, 2nd ed, 2013. https://www-elec.inaoep.mx/~jmram/Digital_Signal_Processing__LI_TAN.pdf
3. ^ Pelgrom, Marcel J.M. Analog-to-Digital Conversion: [E-Book]. 4th edition 2022. Cham: Springer, 2022.
4. ^ A. Wendemuth, E. Andelic, S. Barth, M. Katz, S. Krüger, M. Mamsch, and M. Schafföner, Grundlagen der digitalen Signalverarbeitung: Ein mathematischer Zugang. Berlin, Germany: Springer-Verlag, 2006

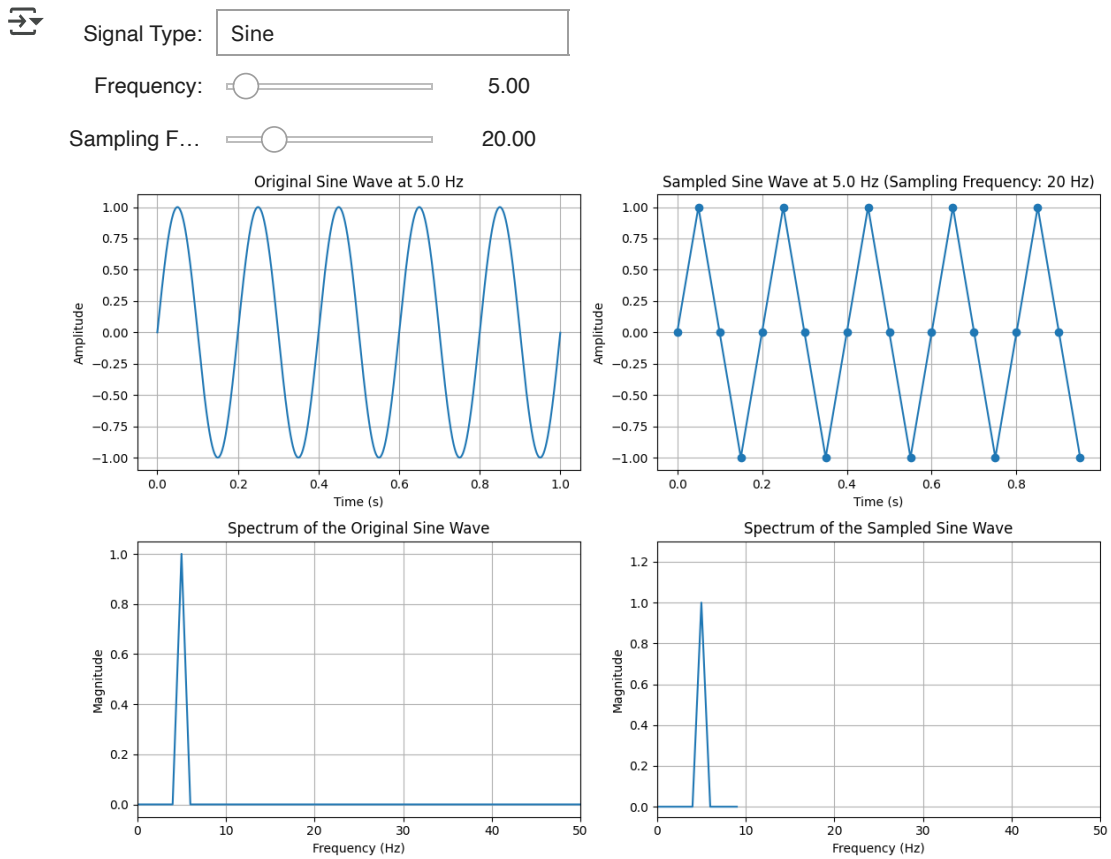
Exercise 1

Generate signals (sine, square, triangle) and display their spectra individually. Display the spectrum of a sampled signal under varying sampling frequencies. Compare with the original signal.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
import ipywidgets as widgets
from IPython.display import display
```

```
# Parameters for signal generation
duration = 1.0  # Signal duration in seconds
original_sampling_frequency = 10000  # high sampling frequency to approximate a
sampling_frequency = 20  # initial lower sampling frequency for comparison

# Time vectors for the original high rate and variable lower rate
t_high = np.linspace(0, duration, int(original_sampling_frequency * duration),

# Manual signal generation functions
def generate_sine_signal(frequency, t):
    return np.sin(2 * np.pi * frequency * t)

def generate_triangle_signal(frequency, t):
    period = 1 / frequency
    return 2 * np.abs(2 * ((t / period) - np.floor((t / period) + 0.5))) - 1

def generate_square_signal(frequency, t):
    return np.sign(np.sin(2 * np.pi * frequency * t))

# Function to calculate and display both signals and their spectra
def draw_signals_and_spectra(signal_type, frequency, sampling_frequency):
    sampling_frequency = int(sampling_frequency)
    t = np.linspace(0, duration, int(sampling_frequency * duration), endpoint=F

    if signal_type == 'Sine':
        signal_high = generate_sine_signal(frequency, t_high)
        signal = generate_sine_signal(frequency, t)
    elif signal_type == 'Triangle':
        signal_high = generate_triangle_signal(frequency, t_high)
        signal = generate_triangle_signal(frequency, t)
    elif signal_type == 'Square':
        signal_high = generate_square_signal(frequency, t_high)
        signal = generate_square_signal(frequency, t)

    # Display of the original and sampled signals
    fig, axs = plt.subplots(2, 2, figsize=(12, 8))

    # Original signal (high sampling frequency)
    axs[0, 0].plot(t_high, signal_high, label='Original Signal')
    axs[0, 0].set_title(f'Original {signal_type} Wave at {frequency} Hz')
    axs[0, 0].set_xlabel('Time (s)')
    axs[0, 0].set_ylabel('Amplitude')
    axs[0, 0].grid(True)

    # Sampled signal
    axs[0, 1].plot(t, signal, marker='o')
    axs[0, 1].set_title(f'Sampled {signal_type} Wave at {frequency} Hz (Samplir
    axs[0, 1].set_xlabel('Time (s)')
    axs[0, 1].set_ylabel('Amplitude')
    axs[0, 1].set_ybound(-1.1, 1.1)
    axs[0, 1].grid(True)

    # Spectrum of the original signal
    signal_fft_high = fft(signal_high)
    freqs_high = fftfreq(len(signal_high), 1/original_sampling_frequency)
    axs[1, 0].plot(freqs_high[:len(freqs_high)//2], 2.0/len(signal_high) * np.a
    axs[1, 0].set_title(f'Spectrum of the Original {signal_type} Wave')
    axs[1, 0].set_xlabel('Frequency (Hz)')
    axs[1, 0].set_ylabel('Magnitude')
    axs[1, 0].set_xbound(0, 50)
    axs[1, 0].grid(True)

    # Spectrum of the sampled signal
    signal_fft = fft(signal)
    freqs = fftfreq(len(signal), 1/sampling_frequency)
```

```
    axs[1, 1].plot(freqs[:len(freqs)//2], 2.0/len(signal) * np.abs(signal_fft[:
    axs[1, 1].set_title(f'Spectrum of the Sampled {signal_type} Wave')
    axs[1, 1].set_xlabel('Frequency (Hz)')
    axs[1, 1].set_ylabel('Magnitude')
    axs[1, 1].set_xbound(0, 50)
    axs[1, 1].set_ybound(-0.05, 1.3)
    axs[1, 1].grid(True)

    plt.tight_layout()
    plt.show()

# Interactive widget for control
interactive_chart = widgets.interactive(
    draw_signals_and_spectra,
    signal_type=widgets.Dropdown(options=['Sine','Triangle', 'Square'], value='
    frequency=widgets.FloatSlider(value=5.0, min=1.0, max=50.0, step=1, descrip
    sampling_frequency=widgets.FloatSlider(value=sampling_frequency, min=1, max
)

display(interactive_chart)
```



1. Compare the harmonic content and spectral distribution of a sine and a square signal. What are the main differences in their spectra and why?

Answer:

- Harmonics are the residue of a signal which are caused while composing the signals with multiple fundamental frequency. When these multiple frequencies do not cancel out completely, it leads to harmonics.
- A sine wave contains only its fundamental frequency, which is reflected as a single peak in the frequency spectrum, with no harmonics.
- In contrast, square and triangle waves consist of the fundamental frequency and several harmonics. This results in multiple peaks in their frequency spectra, corresponding to these harmonic components.
- This imperfection is seen mainly in the square wave in the frequency domain. In the frequency spectrum, we can see small peaks of harmonics after the highest peak of the original signal. Another observation is that the amplitude of the harmonics decreases as the harmonic number increases. It contains odd harmoins.

2. How does changing the sampling frequency affect the spectrum of the sampled signal compared to the original signal? Provide examples with different sampling frequencies and explain observed phenomena such as aliasing.

Considering $f_m = 10Hz$; f_m is the highest requency component; f_s is the sampling frequency

Answer: If the $f_s < 2f_m$, then the highest frequency component of the original signal cannot be seen in the sampled frequency spectrum, and the aliasing effect takes place and misrepresents the highest frequency component as a lower frequency. That is when $f_s = 5, 10, 15Hz.$

Sine : $f_s > f_m$

- To see the f_m of the signal in the frequency spectrum, f_s must be slightly higher than $2f_m$ not exactly $f_s = 2f_m$; $f_s = 24Hz.$
- No harmoinc compontes are seen.

Triangle : $f_s > f_m$

- To see the f_m of the signal in the sampled frequency spectrum, f_s must be slightly higher than the $2f_m$, not exactly $f_s = 2f_m$, that is $f_s = 24Hz$. Harmonics are present in this case.
- To see all frequency components of the signal in the sampled frequency spectrum with no harmonics, then $f_s = 8f_m$ that is $f_s = 80Hz.$
- Harmoincs are seen when $f_s = 3f_m, 5f_m, 7f_m, 9f_m$ but not for even multiples, that is, $f_s = 4f_m, 6f_m, 8f_m, 10f_m.$
- Amplitude of harmonics is low/less compared to square. That is because the $amplitude \propto 1/(n^2)$, where n is the number of harmoinc.

Square: $f_s > f_m$

- To see the f_m of the signal in the sampled frequency spectrum, f_s must be slightly higher than the $2f_m$, not exactly $f_s = 2f_m$, that is $f_s = 24Hz$. Harmonics are

present in this case.

- To see all frequency components of the signal in the sampled frequency spectrum, then $f_s = 7f_m$, that is $f_s = 70Hz$.
- Harmoinc are seen for both odd and even multiples, that is, $f_s = 3f_m, 4f_m, 5f_m, 6f_m, 7f_m, 8f_m, 9f_m, 10f_m$.
- Amplitude of harmonics is high compared to triangular. That is because the $amplitude \propto 1/n$, where n is the number of harmoinc.

For all the waves, the highest frequency component in the sampled frequency spectrum is seen at $f_s = 24Hz$. A sine wave has no harmonics, for a triangle wave, harmonics appear at odd multiples, and for a square wave, harmonics appear at odd and even multiples.

3. Considering the Nyquist Theorem, what is the minimum sampling frequency at which you could accurately reconstruct each signal (sine, square, triangle) from its sampled version? Explain your reasoning with reference to the spectral content of each signal.

Answer: According to the theorem, to reconstruct any signal, the sampling frequency must be greater than or equal to twice the highest frequency component: $f_s \geq 2f_{\mathrm{m}}$.

That is, for a signal with a maximum frequency component of 10 Hz, the minimum sampling frequency would be: $f_s = 24Hz$.

However, as observed in the last exercise on advanced reconstruction, the minimum sampling frequency at which you can accurately reconstruct each signal is as follows:

Sine wave $f_s \geq 2f_m$ i.e., $20Hz$: This is possible because the sine wave consists of a single frequency component and therefore has no harmonics.

Triangle wave: $f_s \geq 5f_m$ i.e., $40 - 50Hz$: The sampling frequency is about 5 times the fundamental frequency due to the presence of multiple fundamental frequencies and harmonics. Also, the harmonic amplitudes $(\sim 1/n^2)$are lower than those of a square wave.

Square wave: $f_s \geq 8 - 10f_m$ i.e.,$80 - 100Hz$: The sampling frequency is 8 to 10 times the fundamental frequency because of multiple fundamental frequencies and harmonics. Additionally, the harmonic amplitudes $(\sim 1/n)$ are higher than those of a triangular wave.

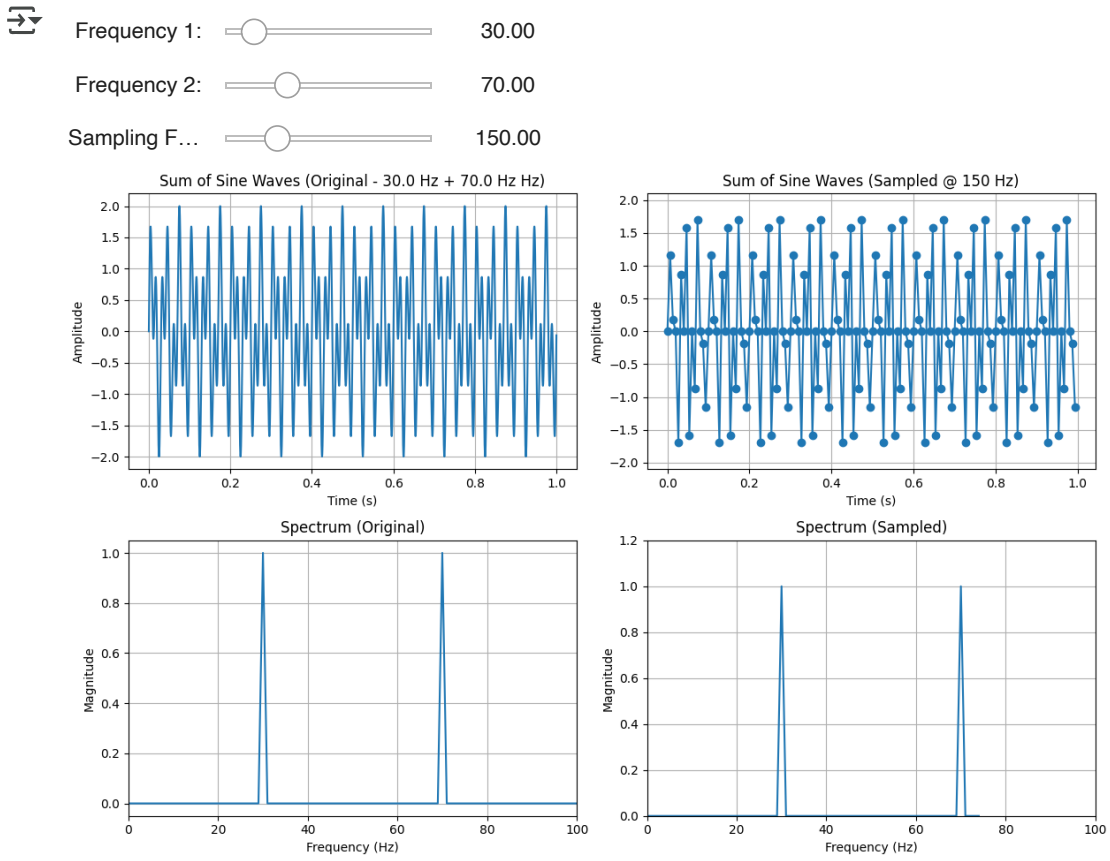
Exercise 2

Generate an overlaid signal (from 2 additively combined sinusoidal signals) and display it in the time and frequency domains. Sample the signal at three different frequencies and display the sampled values. Compare with the original signal and explain the differences.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
import ipywidgets as widgets
from IPython.display import display

# Parameters for signal generation
duration = 1.0  # Signal duration in seconds
original_sampling_frequency = 10000  # High-resolution reference signal

# Time vector for high-resolution signal
t_high = np.linspace(0, duration, int(original_sampling_frequency * duration),

# Manual sine signal generator
```

```
# Handle sine signal generation
def generate_sine_signal(frequency, t):
    return np.sin(2 * np.pi * frequency * t)

# Plotting function for signals and spectra
def draw_signals_and_spectra(title_prefix, freqs_description, sampling_frequenc
    sampling_frequency = int(sampling_frequency)
    t = np.linspace(0, duration, int(sampling_frequency * duration), endpoint=F

    # Sampled signal based on global signal
    global signal, signal_high

    # Plot setup
    fig, axs = plt.subplots(2, 2, figsize=(12, 8))

    # High-res signal
    axs[0, 0].plot(t_high, signal_high)
    axs[0, 0].set_title(f'{title_prefix} (Original - {freqs_description} Hz)')
    axs[0, 0].set_xlabel('Time (s)')
    axs[0, 0].set_ylabel('Amplitude')
    axs[0, 0].grid(True)

    # Sampled signal
    axs[0, 1].plot(t, signal, marker='o')
    axs[0, 1].set_title(f'{title_prefix} (Sampled @ {sampling_frequency} Hz)')
    axs[0, 1].set_xlabel('Time (s)')
    axs[0, 1].set_ylabel('Amplitude')
    axs[0, 1].set_ybound(-2.1, 2.1)
    axs[0, 1].grid(True)

    # FFT of high-res signal
    signal_fft_high = fft(signal_high)
    freqs_high = fftfreq(len(signal_high), 1/original_sampling_frequency)
    axs[1, 0].plot(freqs_high[:len(freqs_high)//2], 2.0/len(signal_high) * np.a
    axs[1, 0].set_title('Spectrum (Original)')
    axs[1, 0].set_xlabel('Frequency (Hz)')
    axs[1, 0].set_ylabel('Magnitude')
    axs[1, 0].set_xbound(0, 100)
    axs[1, 0].grid(True)

    # FFT of sampled signal
    signal_fft = fft(signal)
    freqs = fftfreq(len(signal), 1/sampling_frequency)
    axs[1, 1].plot(freqs[:len(freqs)//2], 2.0/len(signal) * np.abs(signal_fft[:
    axs[1, 1].set_title('Spectrum (Sampled)')
    axs[1, 1].set_xlabel('Frequency (Hz)')
    axs[1, 1].set_ylabel('Magnitude')
    axs[1, 1].set_xbound(0, 100)
    axs[1, 1].set_ybound(0, 1.2)
    axs[1, 1].grid(True)

    plt.tight_layout()
    plt.show()

# Update function triggered by widgets
def update_plots(freq1, freq2, sampling_rate):
    global signal, signal_high
    t = np.linspace(0, duration, int(sampling_rate * duration), endpoint=False)

    # Generate high-resolution signal (sum of two sine waves)
    signal1_high = generate_sine_signal(freq1, t_high)
    signal2_high = generate_sine_signal(freq2, t_high)
    signal_high = signal1_high + signal2_high

    # Generate sampled signal
    signal1 = generate_sine_signal(freq1, t)
    signal2 = generate_sine_signal(freq2, t)
```

```
    signal2 = generate_sine_signal(freq2, t)
    signal = signal1 + signal2

    draw_signals_and_spectra("Sum of Sine Waves", f"{freq1} Hz + {freq2} Hz", s

# Widgets
freq1_widget = widgets.FloatSlider(value=30, min=1, max=250, step=10, descripti
freq2_widget = widgets.FloatSlider(value=70, min=1, max=250, step=10, descripti
sampling_widget = widgets.FloatSlider(value=150, min=5, max=600, step=5, descri

# Interactive control
interactive_plot = widgets.interactive(update_plots, freq1=freq1_widget, freq2=
display(interactive_plot)
```



1. How does changing the sampling frequency affect the sampled signal compared to the original signal?

Answer :

- If $f_s < (f1_m + f2_m)$: Aliasing effect can be seen when high-frequency components are misrepresented as lower-frequency components.

  Example : $f1_m = 30Hz; f2_m = 20Hz; f_s = 40Hz$ Here, the sampled signal—both in the time and frequency domains—does not resemble the original signal. The frequency domain shows aliasing and misinterprets the second frequency component at 10 Hz instead of 20 Hz. The time-domain sampled signal looks like a triangular wave, which is not the original signal. This happens because $f_s < 2(f1_m + f2_m)$

- If $f_s \geq 2max(f1_m, f2_m)$: No aliasing effect; the highest frequency components of both signals can be seen.

  Example : $f1_m = 30Hz; f2_m = 20Hz; f_s = 100Hz$ Here, the sampled signal—both in the time and frequency domains—resembles the original signal, as $f_s >> 2(f1_m + f2_m)$. Two frequency components are shown exactly at 20 Hz and 30 Hz.

2. How does the frequency spectrum of the signal change when the sum of the frequencies of the two sinusoidal signals is close to half of the sampling frequency?

Answer : That is $(f1_m + f2_m) = f_s/2$:

Example 1 : $f1_m = 20Hz; f2_m = 30Hz; f_s/2 = 50Hz$

Example 2 : $f1_m = 150Hz; f2_m = 150Hz; f_s/2 = 300Hz$

For the both the examples , no frequency component is seen in the sampled signal's frequency domain. The sampled signal in the time domain appears as a straight line along the x-axis, indicating no variation—this suggests that the signal has been completely misrepresented due to critical aliasing or cancellation.

3. What are the effects of choosing different sampling frequencies on the ability to distinguish the two overlaid sinusoidal signals in the sampled signal?

Answer : Considering: $f1_m = 30Hz; f2_m = 70Hz$

- $f_s = 40Hz$; $f_s < 2(min(f1_m, f2_m))$: Aliasing may occur, neither $f1_m$ nor $f2_m$ may be clearly visible in the spectrum.
- $f_s = 60Hz; f_s > 2(min(f1_m, f2_m))$: Aliasing may occur, but not sufficient to guarantee the visibility of both components; only the lower frequency component may be visible depending on spectral overlap and resolution.
- $f_s = 140Hz; f_s > 2(max(f1_m, f2_m))$: No aliaing, both $f1_m$ and $f2_m$ can be seen in the spectrum
- $f_s = 200Hz; f_s = 2(f1_m + f2_m)$: No aliaing, both $f1_m$ and $f2_m$ can be seen in the spectrum
- $f_s = 250Hz; f_s > 2(f1_m + f2_m)$: No aliaing, both $f1_m$ and $f2_m$ can be seen in the spectrum

To satisfy the Nyquist–Shannon sampling theorem when dealing with a sum of two sinusoidal signals, the sampling frequency must be greater than twice the highest frequency component, i.e., $f_s > 2(max(f1_m, f2_m))$

Exercise 3

Apply different window functions (Hamming, rectangle, Hann, etc.) to the input sinusoid with a frequency between 1 and 20 Hz, and compare the resulting signals in the time and frequency domains.

```
from scipy.signal.windows import boxcar
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from scipy.signal import square, sawtooth
from scipy.fft import fft, fftfreq
import ipywidgets as widgets
from IPython.display import display
def generate_sine_signal(freq, t):
    return np.sin(2 * np.pi * freq * t)
def plot_sine_wave(freq, window_type, show_high_res, show_DFT):
    # Time parameters
    sampling_rate = 1024  # Sampling rate per second
    duration = 1          # Duration in seconds
    N = 1024          # Number of DFT points
    t = np.linspace(0, duration, int(sampling_rate * duration), endpoint=False)

    # Generate sine wave
    sine_wave = generate_sine_signal(freq, t)

    # Apply window function
    if window_type == 'Hann':
        window = np.hanning(N)
    elif window_type == 'Rectangle':
        window = boxcar(len(sine_wave))
        window[:len(sine_wave)//4] = 0
        window[3*len(sine_wave)//4:] = 0
    elif window_type == 'Bartlett':
        window = np.bartlett(N)
    elif window_type == 'Blackman':
        window = np.blackman(N)
    elif window_type == 'Hamming':
        window = np.hamming(N)
    elif window_type == 'Kaiser':
        window = np.kaiser(N, 14)  # Beta parameter set to 14 for Kaiser

    windowed_sine_wave = sine_wave * window

    # DFT calculations for interactive display
    sine_wave_dft = fft(sine_wave, n=N)
    windowed_sine_wave_dft = fft(windowed_sine_wave, n=N)

    # Normalized magnitude of DFT
    magnitude_dft = np.abs(sine_wave_dft) / N
    magnitude_windowed_dft = np.abs(windowed_sine_wave_dft) / N

    # High-resolution DFT
    high_res_N = 10 * N
    if show_high_res:
        sine_wave_high_dft = fft(sine_wave, n=high_res_N)
        windowed_sine_wave_high_dft = fft(windowed_sine_wave, n=high_res_N)
        high_res_frequency_axis = np.fft.fftfreq(high_res_N, 1/sampling_rate)
        magnitude_high_dft = np.abs(sine_wave_high_dft) / high_res_N * 10
        magnitude_windowed_high_dft = np.abs(windowed_sine_wave_high_dft) / hig

    # Plotting
    plt.figure(figsize=(14, 7))

    # Time-domain plot
    plt.subplot(1, 2, 1)
```

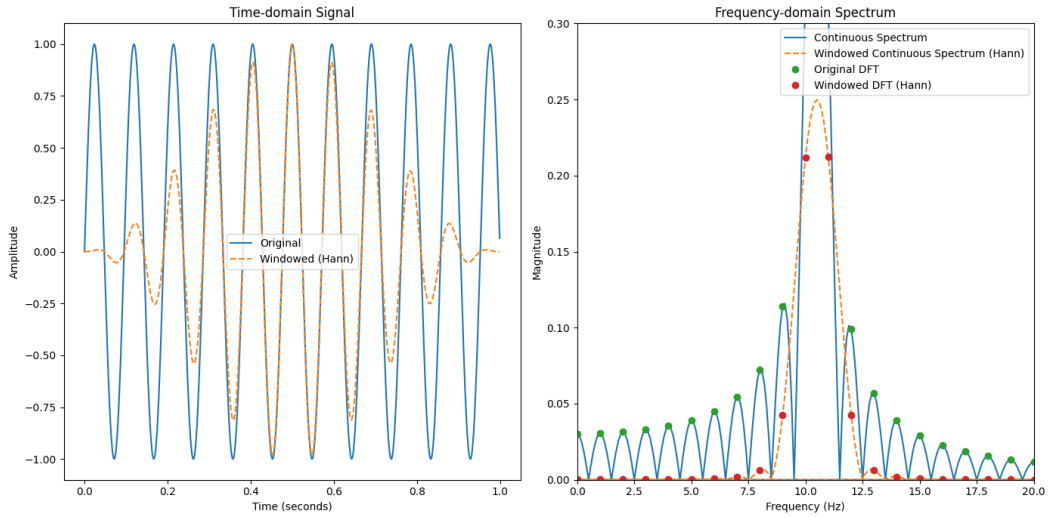
```
    plt.subplot(1, 2, 1)
    plt.plot(t, sine_wave, label='Original')
    plt.plot(t, windowed_sine_wave, label=f'Windowed ({window_type})', linestyl
    plt.title('Time-domain Signal')
    plt.xlabel('Time (seconds)')
    plt.ylabel('Amplitude')
    plt.legend()

    # Frequency-domain plot
    plt.subplot(1, 2, 2)
    if show_high_res:
        plt.plot(high_res_frequency_axis, magnitude_high_dft, label='Continuous
        plt.plot(high_res_frequency_axis, magnitude_windowed_high_dft, label=f'
    if show_DFT:
        plt.plot(np.fft.fftfreq(N, 1/sampling_rate), magnitude_dft, 'o', label=
        plt.plot(np.fft.fftfreq(N, 1/sampling_rate), magnitude_windowed_dft, 'o
    plt.title('Frequency-domain Spectrum')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.xlim(0, 20)
    plt.ylim(0, 0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

# Widgets
freq_slider = widgets.FloatSlider(value=10.5, min=1, max=20, step=0.1, descript
window_selector = widgets.Dropdown(options=['Hann', 'Rectangle', 'Bartlett', 'B
show_high_res_check = widgets.Checkbox(value=False, description='High-resolutio
show_DFT_check = widgets.Checkbox(value=True, description='DFT')


# Interaction
widgets.interactive(plot_sine_wave, freq=freq_slider, window_type=window_select

display(widgets.interactive(plot_sine_wave, freq=freq_slider, window_type=windo
```

Frequency ... 10.50

Window: Hann

High-resolution Spectrum

DFT



1. What is the effect of applying different window functions on the spectrum of a signal?

Answer: As seen in the frequency domain spectrum, the sine wave processed through the DFT without a window function exhibits spectral leakage and inaccurate peaks. Applying window functions before the DFT causes significant changes, reduces spectral leakage, and provides more accurate peaks.

- **Rectangle window**: Ractangular has no tapering, resulting in discotinuties at the edges. It provides the narrowest main lobe which provides best frequency resolution. It also introduces high sidelobes which wrenches nearby frequency, causing notable spectral leakage.
- **Hann window**: With smooth tapering(reducing edge effect), Hann has the moderate main lobe with lower side lobes, which creates less leakage but lower resolution.
- **Hamming window**: Hamming(gradual tapering) also gives moderates main lobe with slightly lower side lobes than Hann,causing a balance between leakage and resolution.
- **Bartlett window**: Bartlett (linear taper) introduces wider main lobe with moderate side lobes which cause moderate leakage.
- **Blackman window**: Blackman has a very gradual tapering, which produces widest main lobe with very low side lobes.It offers best leakage suppression at the cost of low resolution.
- **Kaiser window**: Kaiser window provides best flexibilty in terms of leakage and resolution. Because of it adjustable smooth tapering, it can generate adjustable main lobe width and adjustable side lobe level.

2. Discuss which window function is suitable for identifying amplitude peaks compared to frequency peaks.

Answer:
For identifying amplitude peak

- Hamming is suitable for identifying amplitude peaks because of their low side lobes levels and minimized spectral leakage. It keeps reasonably narrow main lobe , so maintain amplitude very well.

For identifying frequency peaks

- Hann window is best for identifying frequency peaks because of their narrower main lobes, which fix closely spaced frequency and increase frequency resolution.

3. What happens when the frequency of the signal exactly matches a multiple of the DFT sampling frequency?

Answer: If the sine wave's frequency exactly matches one of the DFT bins (for some integer k) i.e., $f = k \times (f_s/N)$; where k index of frequency bin, f_s is the sampling rate and N is the number of DFT points.

- Spectral energy will concentrate at a single frequency bin(k capture all the power).
- Spectral leakage doesn’t occur, and therefore window functions are not necessary in this case.

Exercise 4

In the given sound file **fakecar.wav**, a simmulated car with a honking horn passes a stationary microphone which records the sound.

- Using the Doppler effect, determine the speed of the car and the frequency of the honking horn.

The second sound file **realcar.wav** contains a real life scenario of a car passing by, while a person is playing saxophone.

- determine the speed of the car
- determine the note (A, B, C, ...) the saxophone is playing.

---

help 1: concentrate on frequencies < 1000 Hz

help 2: use a good window function

```
# --- Required Libraries ---
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.fft import fft
from scipy.signal import find_peaks
from IPython.display import display
import ipywidgets as widgets
import gdown

# --- Step 1: Download WAV file from Google Drive ---
file_id = "14ZkpM1ASvAlCuNRf5xJFAUfBOPsdzzIz"
url = f"https://drive.google.com/uc?id={file_id}"
filename = "doppler_audio.wav"
gdown.download(url, filename, quiet=False)

# --- Step 2: Load WAV and convert to mono if needed ---
sampling_rate, data = wavfile.read(filename)
if data.ndim > 1:
    data = data[:, 0]  # Convert to mono

duration = len(data) / sampling_rate

def update_fft(start_time=0, end_time=None, start_freq=0, end_freq=None):
    if end_time is None or end_time > duration:
        end_time = duration
    if end_freq is None or end_freq > sampling_rate / 2:
        end_freq = sampling_rate / 2

    start_sample = int(start_time * sampling_rate)
    end_sample = int(end_time * sampling_rate)
    segment = data[start_sample:end_sample]

    if len(segment) == 0:
        print("Empty audio segment.")
        return

    # --- Apply Hanning window ---
    window = np.hanning(len(segment))
    windowed_segment = segment * window

    #     FFT
```

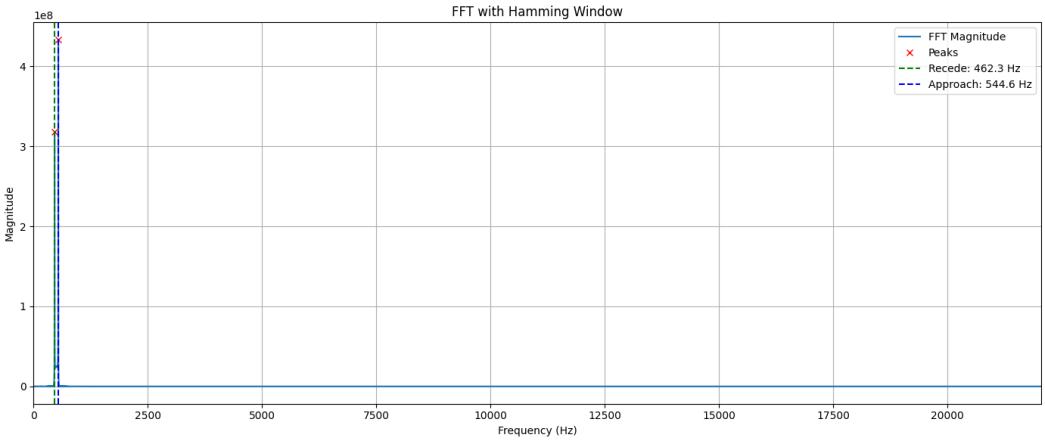
```
# --- FFT ---
segment_fft = fft(windowed_segment)
freqs_fft = np.fft.fftfreq(len(segment), 1/sampling_rate)
magnitude_fft = np.abs(segment_fft)

half_n = len(freqs_fft) // 2
freqs = freqs_fft[:half_n]
magnitude = magnitude_fft[:half_n]

# --- Peak detection ---
peaks, _ = find_peaks(magnitude, height=np.max(magnitude) * 0.3, distance=20
dominant_freqs = freqs[peaks]
dominant_mags = magnitude[peaks]

sorted_indices = np.argsort(dominant_mags)[::-1]
top_freqs = dominant_freqs[sorted_indices][:2]
top_freqs = np.sort(top_freqs)  # Assume recede then approach

# --- Doppler Calculation ---
v_sound = 343  # m/s
if len(top_freqs) < 2:
    print("Not enough peaks to estimate speed.")
    return

f_recede, f_approach = top_freqs
f_source = (f_recede + f_approach) / 2

def estimate_speed(f_s, f_a, f_r):
    v_a = v_sound * ((f_a / f_s) - 1)
    v_r = v_sound * (1 - (f_r / f_s))
    return (v_a + v_r) / 2

speed = estimate_speed(f_source, f_approach, f_recede)

# --- Frequency to musical note ---
def freq_to_note(freq):
    A4 = 440
    notes = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B'
    if freq == 0:
        return "N/A"
    n = int(round(12 * np.log2(freq / A4)))
    note_index = (n + 9) % 12
    octave = 4 + ((n + 9) // 12)
    return f"{notes[note_index]}{octave}"

note = freq_to_note(f_source)

# --- Plot ---
plt.figure(figsize=(14, 6))
plt.plot(freqs, magnitude, label='FFT Magnitude')
plt.plot(dominant_freqs, dominant_mags, 'rx', label='Peaks')
plt.axvline(f_recede, color='green', linestyle='--', label=f'Recede: {f_rece
plt.axvline(f_approach, color='blue', linestyle='--', label=f'Approach: {f_a
plt.title('FFT with Hamming Window')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.xlim(start_freq, end_freq)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()



# --- Print Results ---
```

```
    print(f"Estimated Speed: {speed:.2f} m/s ({speed * 3.6:.2f} km/h)")
    print(f"Estimated Horn Frequency: {f_source:.2f} Hz")
# --- Step 4: Interactive Widgets ---
start_time_slider = widgets.FloatSlider(value=0, min=0, max=duration, step=0.1,
end_time_slider = widgets.FloatSlider(value=duration, min=0, max=duration, step=
start_freq_slider = widgets.FloatSlider(value=0, min=0, max=sampling_rate / 2, s
end_freq_slider = widgets.FloatSlider(value=sampling_rate / 2, min=0, max=sampli

ui = widgets.VBox([
    widgets.HBox([start_time_slider, end_time_slider]),
    widgets.HBox([start_freq_slider, end_freq_slider])
])
out = widgets.interactive_output(update_fft, {
    'start_time': start_time_slider,
    'end_time': end_time_slider,
    'start_freq': start_freq_slider,
    'end_freq': end_freq_slider
})
display(ui, out)
```

```
Downloading...
From: https://drive.google.com/uc?id=14ZkpM1ASvAlCuNRf5xJFAUfBOPsdzzIz
To: /content/doppler_audio.wav
100%|██████████| 202k/202k [00:00<00:00, 17.5MB/s]
<ipython-input-10-787117dbe32a>:18: WavFileWarning: Chunk (non-data) not un
  sampling_rate, data = wavfile.read(filename)
```

Start Time: 0.00 End Time: 2.28

Start Freq: 0.00 End Freq: 22050.00



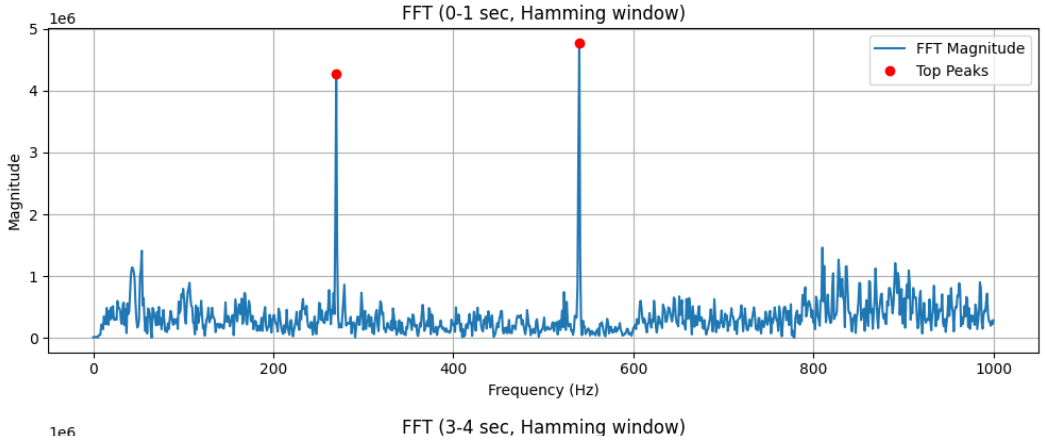

```
Estimated Speed: 28.04 m/s (100.93 km/h)
Estimated Horn Frequency: 503.48 Hz
```

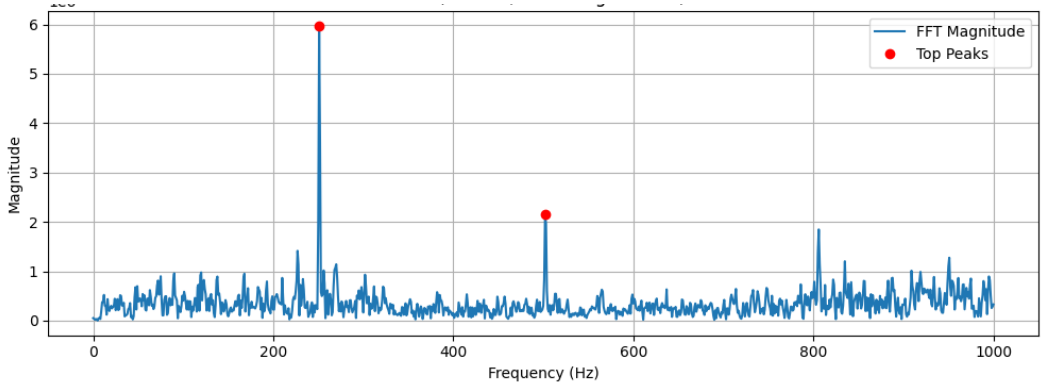
```
# --- Required Libraries ---
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.fft import fft
from scipy.signal import find_peaks
import gdown  # For downloading from Google Drive
```

```
# --- Download WAV file ---
url = "https://drive.google.com/uc?id=1tlSjtGUgvyK5ZeuyHvg3UbWnfFkycLYs"
filename = "downloaded_audio.wav"
gdown.download(url, filename, quiet=False)

# --- Load WAV and convert to mono ---
sampling_rate, data = wavfile.read(filename)
if data.ndim > 1:
    data = data[:, 0]  # Convert to mono
duration = len(data) / sampling_rate


# --- FFT Analysis Function ---
def analyze_segment(start_time, end_time, max_freq=1000):
    start_sample = int(start_time * sampling_rate)
    end_sample = int(end_time * sampling_rate)
    segment = data[start_sample:end_sample]

    if len(segment) == 0:
        print(f"Segment {start_time}–{end_time}s is empty.")
        return None, None

    # --- Hamming Window ---
    window = np.hamming(len(segment))
    windowed_segment = segment * window

    # --- FFT ---
    segment_fft = fft(windowed_segment)
    freqs = np.fft.fftfreq(len(segment), d=1/sampling_rate)
    magnitude = np.abs(segment_fft)

    # Take only positive frequencies up to max_freq
    mask = (freqs >= 0) & (freqs <= max_freq)
    freqs = freqs[mask]
    magnitude = magnitude[mask]

    # --- Peak Detection ---
    peaks, _ = find_peaks(magnitude, height=np.max(magnitude) * 0.3, distance=2
    peak_freqs = freqs[peaks]
    peak_mags = magnitude[peaks]

    # Sort by magnitude descending
    sorted_idx = np.argsort(peak_mags)[::-1]
    top_freqs = peak_freqs[sorted_idx][:2]
    top_mags = peak_mags[sorted_idx][:2]

    # Plot
    plt.figure(figsize=(10, 4))
    plt.plot(freqs, magnitude, label='FFT Magnitude')
    plt.plot(top_freqs, top_mags, 'ro', label='Top Peaks')
    plt.title(f'FFT ({start_time}–{end_time} sec, Hamming window)')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

    return top_freqs, top_mags

# --- Analyze segments 0–1 sec and 3–4 sec ---
freqs_0_1, mags_0_1 = analyze_segment(0, 1, max_freq=1000)
freqs_3_4, mags_3_4 = analyze_segment(3, 4, max_freq=1000)

# --- Proceed if both segments have results ---
if freqs_0_1 is not None and freqs_3_4 is not None and len(freqs_0_1) >= 2 and
```

```
if freqs_0_1 is not None and freqs_3_4 is not None and len(freqs_0_1) >= 2 and
    # Highest frequencies for Doppler
    f_recede = max(freqs_3_4[0], freqs_3_4[1])
    f_approach = max(freqs_0_1[0], freqs_0_1[1])
    f_source = (f_recede + f_approach) / 2
    v_sound = 343  # m/s

    # Doppler calculation
    v_approach = v_sound * ((f_approach / f_source) - 1)
    v_recede = v_sound * (1 - (f_recede / f_source))
    estimated_speed = (v_approach + v_recede) / 2

    # Second-highest for note
    second_peak_freq = np.mean([freqs_0_1[1], freqs_3_4[0]])

    def freq_to_note(freq):
        A4 = 440
        notes = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B
        if freq <= 0:
            return "N/A"
        n = int(round(12 * np.log2(freq / A4)))
        note_index = (n + 9) % 12
        octave = 4 + ((n + 9) // 12)
        return f"{notes[note_index]}{octave}"

    estimated_note = freq_to_note(second_peak_freq)

    # --- Results ---
    print("=== Results ===")
    print(f"0–1 sec: 1st Peak = {freqs_0_1[0]:.2f} Hz, 2nd Peak = {freqs_0_1[1]
    print(f"3–4 sec: 1st Peak = {freqs_3_4[0]:.2f} Hz, 2nd Peak = {freqs_3_4[1]
    print(f"Used for Doppler Effect: {f_recede:.2f} Hz (recede), {f_approach:.2
    print(f"Estimated Speed: {estimated_speed:.2f} m/s ({estimated_speed * 3.6:
    print(f"Used for Musical Note: {second_peak_freq:.2f} Hz → {estimated_note}

else:
    print("Insufficient peak data in one or both segments.")
```

```
Downloading...
From: https://drive.google.com/uc?id=1tlSjtGUgvyK5ZeuyHvg3UbWnfFkycLYs
To: /content/downloaded_audio.wav
100%|██████████| 746k/746k [00:00<00:00, 17.2MB/s]
```






```
=== Results ===
0–1 sec: 1st Peak = 540.00 Hz, 2nd Peak = 270.00 Hz
3–4 sec: 1st Peak = 251.00 Hz, 2nd Peak = 502.00 Hz
Used for Doppler Effect: 502.00 Hz (recede), 540.00 Hz (approach)
Estimated Speed: 12.51 m/s (45.03 km/h)
Used for Musical Note: 260.50 Hz → C4
```